

Software Architecture: The Neutral AI Service Exchange

University of Oulu, Faculty of ITEE, CSE Research Unit

Contents

2. Technology Stack	3
Language Choices	3
Messaging Architecture	3
3. Service Architecture	4
3.1 Matching Engine	4
3.2 Slice Registry	5
3.3 Agent Gateway	5
3.4 Settlement Service	5
3.5 Transcript Service	5
3.6 Certification Authority	6
3.7 Cross-Market Auditor	7
3.8 Penalty Engine	7
3.9 Reputation Service	8
3.10 Market Segmentation Controller	8
4. Data Architecture	8
Event Sourcing	8
Time-Series Schema	9
Epoch Data Pipeline	9

5. Agent SDK Design	10
5.1 Complexity Estimator	10
5.2 Bidding Client	11
5.3 Quality Verifier	11
5.4 Service Discovery Client	11
5.5 Local Routing Engine	12
6. Integrator Enablement Stack	12
6.1 Slice Composer	12
6.2 Capacity Reporter	12
6.3 Exchange Connector	13
6.4 Certification Dashboard	13
7. Deployment Architecture	13
Edge-Cloud Topology	13
8. Phase-by-Phase Build Plan	14
Phase 0: Build the Integrator Stack (Months 0-6, 3-4 Engineers)	14
Phase 1: Launch the Exchange (Months 6-18, 6-8 Engineers)	15
Phase 2: Grow Supply Through Tooling (Months 12-36, 10-15 Engineers)	15
Phase 3: The Platform Flywheel (Months 24-48+, 20+ Engineers)	15
9. Critical Design Decisions	16
Decision 1: Ascending Clinching, Not VCG or Posted Prices	16
Decision 2: Per-Unit Fee, Not Payment-Based Revenue	16
Decision 3: Exchange Absorbs the Mechanism Executor	16
Decision 4: Event Sourcing for All State Mutations	16
Decision 5: Rust for the Matching Engine, Go for Everything Else	17
Decision 6: NATS for Real-Time, Kafka for Durability	17
10. Resolution of Implementation Challenges	17
11. Requirement Traceability	19
12. Verification Strategy	21
Simulation-Based Verification	21
Integration Testing	21
Production Verification	21

The Bottom Line

21

Classification: Internal — Founding Team **Date:** March 2026

This document specifies the software architecture for the startup's two products: a Neutral AI Service Exchange and an Integrator Enablement Stack. The architecture translates formal mechanism-design theory into concrete, buildable software components.

The companion strategy memo establishes the business case (per-unit-fee neutrality, integrator enablement, four-phase GTM). The use case document shows the user-facing scenario (Alex's personal AI agent economy). This document answers: what software do we actually build, in what order, and with what technology?

2. Technology Stack

Language Choices

Component	Language	Rationale
Matching Engine	Rust	Deterministic latency, no GC pauses. LMAX Disruptor-style ring buffer with single-writer principle. Financial exchanges (IEX, Jane Street) use similar patterns.
Max-flow library	Rust (Go FFI)	Push-relabel algorithm with incremental recomputation. Performance-critical: recomputes on topology changes.
All other backend services	Go	Fast compilation, strong concurrency (goroutines), mature microservice ecosystem. Standard choice for cloud-native services.
Agent SDK	TypeScript + Python + Rust-WASM	TypeScript for browser/Node agents, Python for ML ecosystem (OpenCode, Cline integration), Rust-WASM for embedded runtimes (on-device models).
Dashboard UI	React + TypeScript	Standard full-stack web.

Messaging Architecture

The system uses a dual messaging topology:

NATS (real-time, 1-5ms). Handles the auction hot path: price broadcasts during ascending clinching, agent demand/exit messages, real-time capacity updates from integrators. NATS's pub/sub model with leaf-node topology provides geographic distribution via metro relays. Messages are ephemeral (not persisted).

Kafka (durable, 5-50ms). Handles event sourcing: every state mutation (bid, allocation, payment, capacity update, certification change) is captured as an immutable event. The Kafka log is the system's source of truth and its audit trail. Derived state (PostgreSQL, Redis, TimescaleDB) is rebuilt from the event stream.

Why dual? NATS alone lacks persistence (no audit trail). Kafka alone adds 5-50ms to the auction hot path (unacceptable for a mechanism that must complete in 50-100ms). The architecture separates the allocation path (latency-critical, NATS) from the audit path (durability-critical, Kafka).

3. Service Architecture

3.1 Matching Engine

Purpose. Run the ascending clinching auction per mechanism epoch.

Theory mapping. R2.1 (faithful execution), R2.6 (broadcast agent demand/exit), R1.2 (GS-based clinching on polymatroid).

Algorithm (per epoch, 4 steps): 1. Pre-epoch lock: Slice Registry locks slice attributes and capacities for this epoch. The Matching Engine receives the polymatroid rank function $f(S) = \text{max-flow derived from all active integrators' reported capacities}$. 2. Ascending auction loop (repeats until all demand is met or all agents have exited): - Price announcement: broadcast current price vector p via NATS. - Demand collection: agents submit demand at price p within a 20ms window. An agent stays in while price is below its valuation, exits when price exceeds it. - Clinch computation: Edmonds' greedy algorithm on the polymatroid. For each agent i , clinch quantity = $f(S) - f(S - \{i\})$, where S is the set of remaining agents. Complexity: $O(n \log n + n * \tau_f)$ where τ_f is the rank function evaluation time. - Price increment: $p_s \leftarrow p_s + \delta$ for each overdemanded slice type. 3. Final allocation: once the loop terminates, the cumulative clinch quantities are the final allocations. Payments equal the prices at which each unit was clinched. 4. Transcript emit: produce EpochResult event (epoch_id, bids, allocation, payments, price_trajectory) to Kafka.

Key design: Single-writer process with CPU pinning. In-memory ring buffer (LMAX Disruptor pattern) for bid ingestion. No locks, no context switches on the hot path. Current epoch's order book is entirely in-memory; no persistent state.

HA. Leader election via etcd/Raft. Warm standby replays Kafka event stream to reconstruct state. Failover target: <1 second.

APIs. - NATS: `exchange.{slice_type}.price` (pub, price announcements), `exchange.{slice_type}.demand` (sub, agent demand) - Kafka: produces EpochResult events - gRPC: `GetEpochResult(epoch_id)` for historical queries

Phase. Phase 1. Phase 0 uses posted prices (Slice Registry + Agent Gateway, no Matching Engine).

3.2 Slice Registry

Purpose. Maintain the catalogue of available slices. Agents discover slices here. Enforce epoch-locking of attributes.

Theory mapping. R1.2 (GS3: attributes fixed within epoch), R1.3 (governance capacity bounds intersected with resource feasibility).

Key responsibility: At epoch start, lock all slice attributes (capacity, latency, quality, price floor) so they cannot change mid-auction. This is the software enforcement of the GS3 condition: fixed attributes within each epoch guarantee the gross-substitutes property needed for DSIC.

APIs. - gRPC: RegisterSlice(integrator_id, slice_spec), QuerySlices(filter), GetSliceCatalogue(epochs) - NATS: capacity.{integrator_id}.update(sub, real-time capacity updates between epochs) - Internal: LockEpoch() (called by Matching Engine at epoch start)

Data. PostgreSQL for slice definitions; Redis for current capacities and epoch-locked state.

Phase. Phase 0.

3.3 Agent Gateway

Purpose. Entry point for agent SDK connections. Handles authentication, rate limiting, session management, and protocol translation between WebSocket/REST and internal NATS/gRPC.

APIs. - WebSocket: /ws/v1/auction (ascending auction protocol) - REST: /api/v1/slices (discovery), /api/v1/account (balance, history) - Internal: bridges WebSocket messages to NATS subjects

Data. Redis session state.

Phase. Phase 0 (REST only), Phase 1 (add WebSocket auction protocol).

3.4 Settlement Service

Purpose. Execute payments after each epoch. Collect from agents, extract the per-unit fee (phi), disburse to integrators.

Theory mapping. R2.4 (per-unit-fee revenue model). Under domain separation, the exchange's revenue is $\phi \cdot \text{total_allocated_volume}$, maximised by maximising allocation. No incentive to manipulate.

Key design: Double-entry bookkeeping in PostgreSQL. Each epoch settlement is an atomic transaction: - Debit agent accounts by payment amounts - Credit integrator accounts by (payment - ϕ) amounts - Credit exchange revenue account by $\phi \cdot \text{allocated_units}$

APIs. - Internal: consumes EpochResult events from Kafka - gRPC: GetBalance(account_id), GetStatement(account_id, date_range)

Phase. Phase 1.

3.5 Transcript Service

Purpose. Produce the append-only, hash-chained audit trail that makes mechanism deviations detectable.

Theory mapping. R2.5 (append-only transcript), R3.2 (cross-market hash chain).

Hash chain construction:

$$H_t = \text{SHA-256}(\text{epoch_id} \parallel \text{bids_hash} \parallel \text{alloc_hash} \parallel \text{payments_hash} \parallel H_{t-1})$$

Each epoch's hash incorporates the previous epoch's hash, creating a tamper-evident chain. Any retroactive modification of an earlier epoch's transcript invalidates all subsequent hashes.

Cross-market extension (Phase 2+): When multiple exchange instances operate across geographies, the hash incorporates adjacent instances' hashes:

$$H_t = \text{SHA-256}(\text{epoch_id} \parallel \text{bids_hash} \parallel \text{alloc_hash} \parallel \text{payments_hash} \parallel H_{t-1} \parallel \{H_k^t: k \text{ in } \dots\})$$

Data. - PostgreSQL: transcript metadata and hashes (the `audit_transcript` table) - S3: full transcript blobs (serialised bids, allocations, payments)

Schema:

```
audit_transcript
  epoch_id          BIGINT PRIMARY KEY
  exchange_id       UUID
  bids_hash         BYTEA(32)
  alloc_hash        BYTEA(32)
  payments_hash     BYTEA(32)
  prev_hash         BYTEA(32)
  chain_hash        BYTEA(32)    -- H_t
  cross_hashes      JSONB        -- {adjacent_exchange_id: hash}
  blob_ref          TEXT         -- S3 key for full transcript
  created_at        TIMESTAMPTZ
```

APIs. - Internal: consumes `EpochResult` events from Kafka, produces `TranscriptCommit` events - gRPC: `GetTranscript(epoch_range)` (for auditors), `VerifyHash(epoch_id, expected_hash)` (for agents)

Phase. Phase 1.

3.6 Certification Authority

Purpose. Bayesian type tracking for integrators. Manages the certified/warning/decertified lifecycle.

Theory mapping. R3.5 (audit signals with known emission probabilities), R3.6 (decertification when posterior exceeds threshold), R3.7 (forgetting factor for non-stationary types), R3.8 (credibility quality publication).

Algorithm. Geometric log-likelihood-ratio (LLR) discounting:

$$L_t = \rho * L_{t-1} + \log(\text{Pr}(z_t \mid \text{strategic}) / \text{Pr}(z_t \mid \text{honest}))$$

$$\text{posterior} = \text{sigmoid}(L_t + \log(\text{prior_odds}))$$

Where ρ is the forgetting factor (tunable per integrator), z_t is the audit signal from the Cross-Market Auditor, and state transitions are: - Certified $\rightarrow$ Warning when posterior $> \tau_w$ - Warning $\rightarrow$ Decertified when posterior $> \tau_d$ - Decertified $\rightarrow$ Recertifying (with elevated prior, making recovery slower than degradation)

Output. Publishes credibility quality q_j per integrator, consumed by agents via the SDK's Service Discovery Client.

Data. PostgreSQL: integrator certification states and posterior histories.

Phase. Phase 2. (Phase 1 uses simple trust scores via Reputation Service.)

3.7 Cross-Market Auditor

Purpose. Verify bid-allocation-payment consistency, detect coalition deviations.

Theory mapping. R3.2 (cross-market audit), Theorem 1 Corollary 1 (coalition detection requires simultaneous access to all domains).

Detection methods: - *Per-integrator statistical tests (Phase 1):* Compare payment distributions against expected clinching payments for the declared polymatroid. Detects systematic payment inflation. - *Cross-integrator consistency (Phase 2+):* Compare allocation patterns across integrators serving overlapping agent populations. Correlated deviations (e.g., two integrators simultaneously reducing capacity) trigger anomaly signals. - *Coalition detection (Phase 3):* Verify that joint allocation vectors across integrator groups are consistent with independent mechanism execution. Inconsistency flags potential coalition.

Output. Emits `AuditSignal(integrator_id, epoch_id, {compliant, anomalous})` events consumed by the Certification Authority.

Data. ClickHouse for historical audit analytics.

Phase. Phase 2 (per-integrator), Phase 3 (coalition detection).

3.8 Penalty Engine

Purpose. Make deviation unprofitable by imposing costs exceeding the deviation gain.

Theory mapping. R3.3 (slashing deposit), R3.4 (state-dependent penalties for contextual adversaries), R3.9 (coalition penalty exceeds per-member share).

Two modes:

State-independent (Phase 1): Each integrator posts a deposit B at registration. Upon detected deviation, partial or full slashing. Deposit sizing: $B \geq \Delta_{\text{bar}} / \Pr(\text{detected} \mid \text{deviation})$, initially set conservatively and refined from historical data.

State-dependent (Phase 3): For sophisticated adversaries that learn to deviate only in profitable states: fine schedule $p_F(s) = \alpha * \Delta_{\text{bar}}(s)$ with safety margin $\alpha > 1$. Requires estimation of $\Delta_{\text{bar}}(s)$ per observable state from historical bid distributions (ClickHouse analytics).

Data. PostgreSQL deposit ledger.

Phase. Phase 1 (fixed deposits), Phase 3 (adaptive fines).

3.9 Reputation Service

Purpose. Maintain trust scores for agents and integrators.

Theory mapping. Paper 1's asymmetric reputation updates (+0.03 compliance, -0.08 violation).

Data. PostgreSQL trust scores. Phase 2+ integrates with Certification Authority's Bayesian posterior for integrators.

Phase. Phase 1.

3.10 Market Segmentation Controller

Purpose. Detect death spirals and manage agent-integrator matching health.

Theory mapping. Proposition 2 (assortative sorting: high-value agents exit first when credibility degrades, creating self-reinforcing decline).

Detection. Monitors departures per integrator per time window. When departure rate exceeds threshold, triggers: - Enhanced monitoring (more frequent audit signals to Certification Authority) - Agent advisories (surfaced via SDK's Service Discovery Client) - Intervention protocols (route agents to alternative integrators)

Data. TimescaleDB time-series of participation metrics.

Phase. Phase 2.

4. Data Architecture

Event Sourcing

Every state mutation is captured as an immutable event in Kafka before being applied to any derived state. This provides the append-only transcript required by the theory (R2.5) as a natural property of the architecture.

Key event types:

Event	Payload	Producer	Consumers
SliceRegistered	integrator_id, slice_spec, capacity	Slice Registry	Matching Engine, Analytics
CapacityUpdated	integrator_id, slice_id, new_capacity	Exchange Connector	Slice Registry, Matching Engine
BidSubmitted	epoch_id, agent_id, slice_type, valuation	Agent Gateway	Matching Engine
EpochResult	epoch_id, bids[], allocation[], payments[], price_trajectory	Matching Engine	Settlement, Transcript, Analytics

Event	Payload	Producer	Consumers
TranscriptCommit	epoch_id, chain_hash, prev_hash, blob_ref	Transcript Service	Cross-Market Auditor, Certification Authority
AuditSignal	integrator_id, epoch_id, signal	Cross-Market Auditor	Certification Authority
CertificationChange	integrator_id, old_state, new_state, posterior	Certification Authority	Market Segmentation, Slice Registry, SDK
SettlementComplete	epoch_id, payments[], fees[], disbursements[]	Settlement Service	Revenue reporting, Dashboard

Time-Series Schema

TimescaleDB hypertables for operational metrics:

capacity_telemetry (hypertable, partitioned by time)

time	TIMESTAMPTZ
integrator_id	UUID
slice_id	UUID
reported_capacity	INTEGER
actual_utilisation	FLOAT
latency_p50	FLOAT
latency_p99	FLOAT

price_history (hypertable)

time	TIMESTAMPTZ
slice_type	TEXT
geo	TEXT
clearing_price	NUMERIC
volume	INTEGER
n_bidders	INTEGER

Epoch Data Pipeline

The theoretical 7-phase epoch maps to actual data flows:

Phase	Theory	Implementation	Latency
1. Pre-epoch	Slice Constructor computes polymatroid	Capacity Reporter $\square$ NATS $\square$ Slice Registry (lock attributes)	10ms
2. Mechanism	Ascending clinching auction	Matching Engine (Rust, in-memory, NATS broadcast)	50-100ms

Phase	Theory	Implementation	Latency
3. Commitment	Transcript + hash chain	Kafka event ☐ Transcript Service ☐ PostgreSQL	20ms
4. Procurement	Integrator bids at Level 2	Exchange Connector ☐ ☐ integrator's resource procurement	50ms
5. Settlement	Payment collection + disbursement	Settlement Service ☐ ☐ PostgreSQL ledger	20ms
6. Audit	Cross-market consistency check	Cross-Market Auditor ☐ ClickHouse	Seconds (async)
7. Penalty	Slash if triggered	Penalty Engine ☐ PostgreSQL deposit ledger	Seconds (async)

Total synchronous path (Phases 1-5): ~150-200ms. This means the exchange serves tasks with 150-200ms latency budgets (typical for interactive AI workloads). Tasks requiring tighter latency (under 150ms) are handled by the SDK's Complexity Estimator, which routes them locally or to pre-allocated edge capacity without auction overhead. The exchange adds value for tasks where dynamic price discovery justifies the auction latency.

Phases 6-7 run asynchronously; their results inform the next epoch's governance state (one-epoch detection delay, as specified in the theory).

5. Agent SDK Design

The SDK is the demand-side interface. It runs inside each personal AI agent (phone, home server, or cloud) and provides five capabilities. Phase 0 ships the Complexity Estimator, Service Discovery, and posted-price acceptance. Phase 1 adds the Bidding Client (ascending auction), Quality Verifier, and full Local Routing Engine.

5.1 Complexity Estimator

Local-first routing logic, shipped as Rust-compiled WASM for embedding in any runtime:

```
estimate_tier(task) -> LOCAL | EDGE | CLOUD
  features = extract(task.type, task.input_size, task.reasoning_depth,
                    task.latency_requirement)
  if features.reasoning_depth < THRESHOLD_LOCAL
    and features.input_size < local_model_capacity:
    return LOCAL
  if features.latency_requirement < 200ms and edge_available():
```

```
    return EDGE
return CLOUD
```

Thresholds are learned from the agent's history (which tasks succeeded locally vs. needed escalation). This is the component that keeps simple tasks local, ensuring the exchange only handles tasks where market coordination adds value. In Alex's use case, this routes morning planning to the phone, coding to the home server, and only edge/cloud tasks to the exchange.

5.2 Bidding Client

Implements the agent side of the ascending auction:

```
WebSocket /ws/v1/auction:
  Client sends:
    → join(agent_id, slice_type, max_valuation)
    → demand(quantity: 0 or 1)    // exit when price > valuation
  Server sends:
    ← price_announce(current_price)
    ← clinch(quantity, payment)
    ← epoch_end(allocation, payment, transcript_hash)
```

Under DSIC on polymatroids (the theoretical guarantee), the dominant strategy is trivially simple: stay in the auction while the current price is below the agent's true valuation, exit when it exceeds. The SDK implements this automatically. No strategic reasoning required from the agent.

5.3 Quality Verifier

Post-allocation SLA compliance check:

```
verify(allocation, actual_latency, actual_throughput, actual_quality)
  → PASS | FAIL(reason)
```

Results feed back to the Reputation Service (updating the integrator's trust score) and to the agent's local trust model (enabling the agent to build its own preferences over integrators, independent of the platform's certification).

5.4 Service Discovery Client

Queries the Slice Registry for slices matching task requirements:

```
discover_slices(task_type, latency_bound, quality_min, geo)
  → SliceListing[] // filtered by certification status, sorted by price + quality
```

5.5 Local Routing Engine

Orchestrates the full task lifecycle: 1. Complexity Estimator classifies the task 2. If LOCAL: execute on device, done 3. If EDGE or CLOUD: Service Discovery finds slices $\square$ Bidding Client participates in auction $\square$ task executes on allocated slice $\square$ Quality Verifier checks result 4. Fallback: if auction fails (no matching slices or price too high), try next tier or queue for next epoch

6. Integrator Enablement Stack

The supply-side bootstrap. These tools make it easy for infrastructure owners to become integrators without building marketplace capabilities from scratch.

6.1 Slice Composer

Core engine (Rust with Go FFI): - DAG editor: define nodes (compute resources) and edges (service dependencies). GUI for non-technical building operators; CLI/API for developers. - Push-relabel max-flow algorithm with incremental recomputation: when a node's capacity changes, recompute only the affected portion of the flow network (not the full graph). - Node-splitting: convert node-capacitated DAGs to edge-capacitated networks ($v \square v_{in}, v_{out}$ with edge capacity C_v). Required for correct max-flow computation. - GS validation: verify that resulting slices satisfy unit demand (one slice per agent per task), additive separability (no cross-task complementarities), and fixed attributes (locked at epoch start by Slice Registry).

Slice template library. Pre-built templates for common patterns: - edge-inference: single GPU node, latency < 100ms, configurable model - cloud-reasoning: cloud endpoint, latency < 5s, throughput guaranteed - device-edge-cloud: 3-node chain, end-to-end latency < 200ms - test-runner: parallel GPU execution, 4-hour availability windows - agentic-service: capability-based (code review, CI/CD), quality metrics

Output. Slice specification (JSON) ready for registration on the Exchange.

Theory mapping. R1.1 (polymatroidal encapsulation), R1.2 (GS validation), R1.5 (submodular rank function).

6.2 Capacity Reporter

Lightweight agent daemon deployed on each compute node in the integrator's infrastructure: - Collects GPU utilisation, memory usage, network bandwidth, queue depth - Aggregates across nodes to compute current effective capacity per slice type - Publishes updates to Exchange via NATS (through Exchange Connector) - 5-second heartbeat; 3 missed heartbeats trigger automatic capacity reduction

For GPU cooperatives, the Capacity Reporter includes additional functionality: idle-hour detection (reduce reported capacity when the owner is using the GPU), availability scheduling (keyed to patterns like "weekday 9am-5pm" or "gaming hours 7pm-11pm"), and automatic capacity pooling across cooperative members.

Theory mapping. R1.4 (cross-domain capacity tracking), Sub-DAG Manager requirements.

6.3 Exchange Connector

Three deployment modes: - **Library:** Go package imported into integrator's existing Go services - **Sidecar:** standalone Go binary proxying between integrator APIs and Exchange APIs (Kubernetes sidecar pattern) - **Helm chart:** full Kubernetes deployment bundling Capacity Reporter + Exchange Connector + monitoring

APIs proxied: - Slice registration and capacity updates □ Slice Registry - Transcript retrieval and hash verification □ Transcript Service - Settlement and disbursement queries □ Settlement Service - Certification status and governance signals □ Certification Authority

Theory mapping. R2.2 (commitment device interface), R3.2 (transcript submission to Cross-Market Auditor).

6.4 Certification Dashboard

Web UI for integrators to monitor their compliance status:

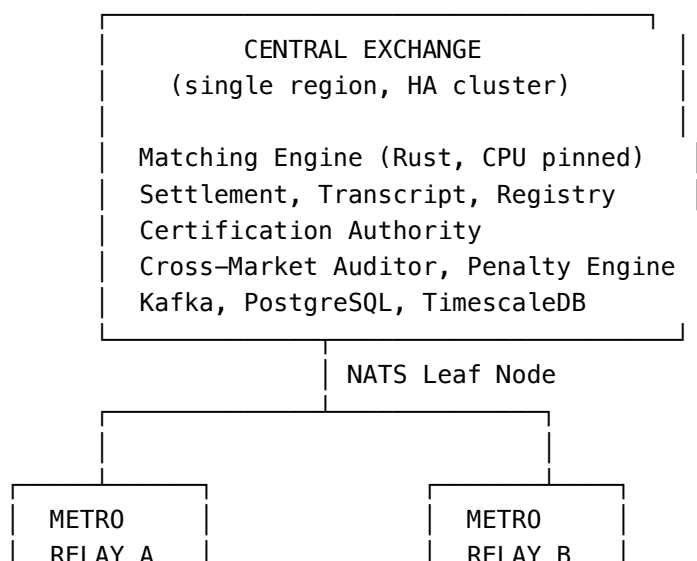
Panels: - **Certification state:** current status (certified/warning/decertified) with posterior trajectory chart showing historical credibility score - **SLA compliance:** actual vs. committed latency, throughput, and quality, with trend lines - **Deposits and penalties:** current deposit balance, slashing history, fine schedule - **Audit timeline:** compliant/anomalous signals per epoch, with drill-down to specific audit findings - **Revenue analytics:** slices allocated, fees earned, utilisation trends, revenue per slice type

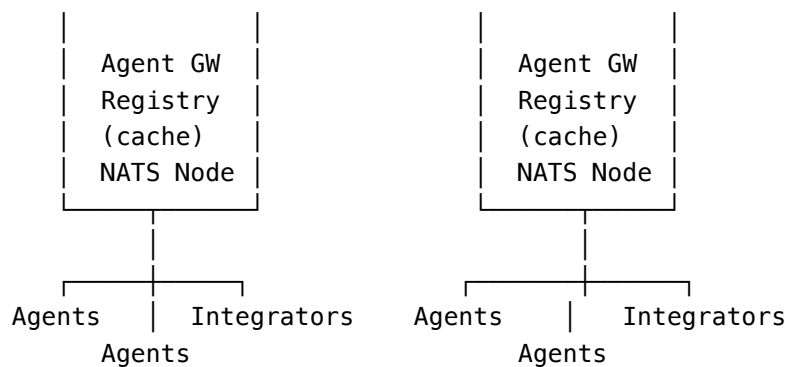
Technology. React + TypeScript frontend, Go backend, TimescaleDB for historical data.

Phase. Phase 2 (full UI). Phase 1 ships a basic status page within the Exchange Connector.

7. Deployment Architecture

Edge-Cloud Topology





Central exchange. Single-region HA cluster (initially one cloud region). Runs the Matching Engine (CPU-pinned, single-writer), all backend services, and the full data platform. All auction computation happens here.

Metro relays (Phase 2+). K3s or KubeEdge clusters at metro points-of-presence. Each relay runs an Agent Gateway (WebSocket termination, reducing round-trip latency for agents), a Slice Registry cache (Redis, for fast discovery queries), and a NATS leaf node (bridging agent messages to the central NATS cluster). Agents connect to the nearest relay; the relay bridges to the central exchange.

HA strategy. The Matching Engine is a single-writer process (no active-active). Failover uses etcd/Raft leader election with a warm standby that continuously replays the Kafka event stream. Failover time target: <1 second. All other services run as standard Go microservices with Kubernetes horizontal pod autoscaling.

Scaling by phase: - Phase 0-1: single central exchange, all connections direct (no relays) - Phase 2: add metro relays for latency-sensitive markets (first relay at beachhead campus) - Phase 3: multiple exchange instances per geography, linked by cross-exchange hash chains for global audit consistency

8. Phase-by-Phase Build Plan

Phase 0: Build the Integrator Stack (Months 0-6, 3-4 Engineers)

Build: - Slice Composer: DAG editor + max-flow engine (Rust), Go wrapper, CLI, basic web UI - Capacity Reporter: Go daemon with GPU telemetry - Mock Exchange: Go service with posted prices (integrator lists slices at fixed price, agents query and accept; no auction). This is the Slice Registry + Agent Gateway without the Matching Engine. - Agent SDK v0.1: Python + TypeScript, discovery + posted-price acceptance only

Co-development. Work with one campus operator to define slice templates for their edge GPU rack (e.g., 4x RTX 4090s). Validate capacity reporting against actual GPU utilisation. Define SLA parameters for test-runner and inference slices.

Revenue. Co-development fees or pilot grant. No transaction revenue.

Theory status. No auction mechanism. Posted prices. No credibility infrastructure (single integrator, bilateral relationship). The formal requirements (R1.1-R3.9) are not yet binding; the market is too small for manipulation to be profitable or detectable.

Phase 1: Launch the Exchange (Months 6-18, 6-8 Engineers)

Build: - Matching Engine (Rust, ascending clinching on polymatroid) - Agent Gateway (Web-Socket + REST) - Settlement Service (Go, double-entry ledger) - Transcript Service (Go, hash-chained audit log) - Penalty Engine v1 (fixed deposits) - Reputation Service v1 (simple trust scores) - Exchange Connector (Go library + sidecar) - Agent SDK v1.0 (all 5 modules)

Launch. Campus operator lists slices on the real exchange. Onboard 1-2 enterprise AI deployments as demand-side agents. Run ascending clinching with broadcast price announcements from day one.

Revenue. Per-unit transaction fees + integrator platform subscriptions.

Theory status. Full DSIC mechanism (ascending clinching). Broadcast commitment via NATS. Fixed deposits. No cross-market audit (single integrator), no Bayesian type tracking (not needed with one integrator). The ascending auction format embeds credibility: agent demand/exit messages are broadcast, so clinch quantities are publicly reconstructible. This is the lightweight commitment device from the strategy memo.

Phase 2: Grow Supply Through Tooling (Months 12-36, 10-15 Engineers)

Build: - Certification Authority (Bayesian type tracking with geometric LLR discounting) - Cross-Market Auditor (per-integrator statistical tests, cross-integrator consistency) - Market Segmentation Controller (death-spiral detection, agent advisories) - Certification Dashboard (full web UI) - Metro relay infrastructure (first relay at beachhead campus) - Cooperative tools (capacity pooling, availability scheduling, revenue splitting) - ClickHouse analytics pipeline

Onboard. Additional building operators, cloud brokers, community GPU cooperatives across multiple metros. Each uses the enablement stack.

Revenue. Growing transaction fees + platform subscriptions + certification fees.

Theory status. Full governance stack. Bayesian type tracking. Cross-market audit for detecting correlated deviations. Two or more competing integrators per service class (pricing converges toward marginal cost with competition; two independent integrators halve monopoly markup).

Phase 3: The Platform Flywheel (Months 24-48+, 20+ Engineers)

Build: - State-dependent penalty estimation (contextual-bandit adversary defence via ClickHouse analytics) - Coalition detection at scale (full cross-market simultaneous verification) - Data products (capacity planning for integrators, demand forecasting for investors, benchmarking for governance) - Multi-geography exchange instances with cross-exchange hash chains - A2A/ACP protocol integration (Google's Agent-to-Agent and Agentic Commerce protocols as agent communication layers)

Revenue. Transaction fees + platform subscriptions + certification fees + data products + regulatory partnership revenue.

Theory status. Full adversarial resilience. State-dependent penalties. Coalition detection. Dynamic credibility with forgetting. The exchange is infrastructure.

9. Critical Design Decisions

Decision 1: Ascending Clinching, Not VCG or Posted Prices

Posted prices are simple but provide no price discovery, no efficiency guarantee, and no DSIC. Sealed-bid VCG is efficient and DSIC but not credible without external commitment (the credibility trilemma from Paper 2). Ascending clinching with broadcast is the only mechanism that is simultaneously DSIC, efficient, and self-committing: agents broadcast their own demand/exit messages, so clinch quantities are publicly reconstructible. Any manipulation produces a detectable inconsistency.

The broadcast uses NATS pub/sub (1-5ms per message), not blockchain consensus (seconds to minutes). This keeps the mechanism within the 100-200ms latency budget for real-time AI services.

Phase 0 starts with posted prices for simplicity. Phase 1 switches to ascending clinching.

Decision 2: Per-Unit Fee, Not Payment-Based Revenue

Under per-unit-fee revenue (ϕ per allocated slice unit), the exchange's profit is $\phi * \text{total_allocated_volume}$. This is maximised by maximising total allocation, which is identical to maximising efficiency. The incentive alignment is exact.

Under payment-based revenue (sum of payments minus costs), the exchange could profit from artificial scarcity: reducing supply raises agent externalities and VCG payments. The credibility trilemma proves this conflict is irresolvable when the operator has any resource stake.

Per-unit-fee also resolves the encapsulation credibility gap: integrators with per-unit-fee revenue are incentivised to report capacity truthfully (more capacity $\rightarrow$ more volume $\rightarrow$ more revenue; over-reporting is detectable via SLA failures).

Decision 3: Exchange Absorbs the Mechanism Executor

The theoretical architecture places the Mechanism Executor inside the integrator, creating an entity that simultaneously controls resources, sees bids, and computes payments. This is the dual-role trust concentration (Challenge 8 from the implementation analysis).

Moving the Mechanism Executor to the Exchange resolves this by structural separation. The Exchange sees bids and computes allocations (but owns no resources). The integrator controls resources and reports capacity (but never sees other integrators' bids). Combined with per-unit-fee, this resolves both execution credibility (Paper 2) and encapsulation credibility (Paper 3 Remark 2).

This also resolves transcript integrity (Challenge 3): the Exchange produces the transcript, not the integrator. An integrator cannot fabricate a transcript for a mechanism it does not run.

Decision 4: Event Sourcing for All State Mutations

Every state change flows through Kafka as an immutable event before being applied to any derived state. The Kafka log IS the append-only transcript required by R2.5. Derived states (PostgreSQL, Redis, TimescaleDB) can be rebuilt by replaying the event stream.

This provides tamper evidence by construction: modifying a past event would require modifying the Kafka log, which is append-only and replicated across brokers. The Transcript Service adds an additional layer by computing SHA-256 hash chains over the event stream.

Decision 5: Rust for the Matching Engine, Go for Everything Else

The Matching Engine must provide deterministic sub-millisecond latency per operation. Rust's zero-cost abstractions, lack of garbage collection, and ownership model are suited to this. The LMAX Disruptor pattern (ring buffer, single-writer, mechanical sympathy) translates well to Rust.

All other services (Settlement, Transcript, Certification, etc.) operate in the 1ms-1s range where Go's simplicity, fast compilation, and goroutine model dominate. Using Rust everywhere would slow development velocity without latency benefit for these services.

Decision 6: NATS for Real-Time, Kafka for Durability

The ascending auction requires sub-10ms broadcast of price announcements. NATS provides this. But NATS messages are ephemeral. The audit trail requires durable, ordered, replayable event storage, which is Kafka's strength.

Using only Kafka would add 5-50ms to the hot path (unacceptable). Using only NATS would sacrifice durability (unacceptable for audit).

10. Resolution of Implementation Challenges

The theoretical architecture analysis identifies 11 implementation challenges. The software architecture addresses each:

#	Challenge	Severity	Resolution
1	Max-flow recomputation under dynamic topology	High	Rust push-relabel with incremental recomputation in Slice Composer. Epoch-locked capacities in Slice Registry prevent mid-auction changes.
2	GS validation at the boundary	High	Slice Composer validates GS conditions at slice definition time. Slice Registry locks attributes at epoch start (enforcing GS3).

#	Challenge	Severity	Resolution
3	Transcript integrity	Critical	Exchange produces the transcript (not the integrator). Kafka event sourcing = append-only by construction. Transcript Service adds SHA-256 hash chains.
4	Deposit sizing under uncertainty	High	Conservative initial deposits (Phase 1). Penalty Engine refines from historical Delta_bar estimates via ClickHouse analytics (Phase 2+).
5	State-dependent penalty estimation	High	Phase 3: ClickHouse historical analysis estimates Delta_bar(s) per observable state. Penalty Engine applies $\alpha * \Delta_bar(s)$ with safety margin.
6	Audit signal calibration	Medium-High	Cross-Market Auditor uses statistical tests with adaptive epsilon estimation. Calibration tests run periodically against known-good epochs.
7	Cross-hash synchronisation	Medium	Phase 1: single exchange, no cross-hash needed. Phase 2+: NATS-based hash exchange between instances, async fallback.

#	Challenge	Severity	Resolution
8	Dual-role trust concentration	Critical	Resolved. Exchange absorbs Mechanism Executor. Integrator retains only encapsulation role. Per-unit-fee ensures truthful capacity reporting.
9	Death spiral feedback loop	Medium-High	Market Segmentation Controller monitors departures per integrator, triggers enhanced monitoring and agent advisories.
10	Epoch boundary synchronisation	Medium	Single exchange synchronises all epochs. Phase 3 multi-exchange: barrier protocol via NATS.
11	Recursive credibility of audit infrastructure	Critical	Resolved institutionally. The Exchange IS the certification authority. Its per-unit-fee incentive alignment provides the credibility grounding. Regulatory partnership adds external backing.

11. Requirement Traceability

Every formal requirement from the trilogy is implemented by at least one software service:

Requirement	Description	Implementing Service(s)
R1.1	Polymatroidal slice encapsulation	Slice Composer (max-flow engine)
R1.2	Gross-substitutes preservation	Slice Composer (validation) + Slice Registry (epoch lock)

Requirement	Description	Implementing Service(s)
R1.3	Governance capacity bounds	Slice Registry (intersect X _{gov} with X _{res} at epoch start)
R1.4	Cross-domain resource procurement	Capacity Reporter + Exchange Connector
R1.5	Submodular rank function	Slice Composer (push-relabel max-flow)
R1.6	Two-level hybrid architecture	Exchange (Level 1) + integrator's own Level-2 participation
R2.1	Faithful mechanism execution	Matching Engine (ascending clinching)
R2.2	Commitment device	Matching Engine (NATS broadcast) + Transcript Service (hash chain)
R2.3	Domain separation	Architecture (Decisions 2-3): Exchange owns no resources
R2.4	Per-unit-fee revenue	Settlement Service (phi extraction)
R2.5	Append-only transcript	Kafka event sourcing + Transcript Service
R2.6	Broadcast demand/exit messages	Agent Gateway + NATS pub/sub
R3.1	No private bid sharing	Matching Engine (bids visible only to Exchange)
R3.2	Cross-market audit	Transcript Service (hash chain) + Cross-Market Auditor
R3.3	Slashing deposit	Penalty Engine (deposit management)
R3.4	State-dependent penalties	Penalty Engine (Phase 3, ClickHouse-backed)
R3.5	Audit signal generation	Cross-Market Auditor (generates) + Certification Authority (consumes)
R3.6	Decertification	Certification Authority (posterior > tau _d)
R3.7	Forgetting factor	Certification Authority (tunable rho)
R3.8	Credibility publication	Certification Authority □ SDK Service Discovery
R3.9	Coalition penalty bound	Penalty Engine (deposit sizing includes coalition factor)

12. Verification Strategy

Simulation-Based Verification

The existing R + targets simulation pipeline provides the empirical validation framework: - **DSIC verification:** Submit synthetic bids with known optimal strategies to the real Matching Engine. Verify allocation and payment match theoretical prediction. - **Learning convergence:** Verify Bayesian type tracker converges at theoretical rates (epsilon-greedy: $O(T^{-1})$, EXP3: $O(T^{-0.5})$) using simulated audit signal streams. - **Death-spiral detection:** Simulate credibility degradation and verify Market Segmentation Controller triggers appropriate advisories.

Integration Testing

- **Transcript verifier:** Standalone tool that replays EpochResult events and checks: (a) allocations feasible on declared polymatroid, (b) payments match clinching rule, (c) hash chain valid, (d) settlement amounts balance. Runs on every epoch in CI/CD.
- **DSIC checker:** Automated test harness submitting truthful vs. strategic bids, verifying truthful bidding weakly dominates.
- **Epoch timing validation:** Measure end-to-end epoch latency under load; alert if synchronous path exceeds 200ms.

Production Verification

- **Canary deployment:** Each new Matching Engine version runs in parallel on a shadow copy of the bid stream. Results compared; divergence triggers alerts.
- **Continuous audit:** Cross-Market Auditor runs on every production epoch. This is not just governance; it is primary production monitoring.
- **SLA tracking:** Quality Verifier feedback from agents fed into TimescaleDB; dashboard alerts on SLA degradation trends.

The Bottom Line

The architecture is two things at once: a financial exchange for AI service slices, and an enablement platform for the entities that supply those slices. The exchange earns per-unit fees on volume. The enablement stack earns subscriptions and onboarding revenue. Neither product requires owning a single GPU.

The key insight is structural: moving the Mechanism Executor from the integrator to the Exchange resolves the trust concentration problem that would otherwise require complex cryptographic commitments. Combined with per-unit-fee revenue, this makes neutrality a natural property of the software architecture, not a policy commitment that might be abandoned.

The build plan is incremental. Phase 0 ships a Slice Composer, a Capacity Reporter, and a mock exchange with posted prices. Phase 1 adds the real auction. Phase 2 adds the governance stack. Phase 3 adds adversarial resilience. At each phase, the architecture is production-complete for the market size it serves. The credibility built at each phase compounds into the next.